

AI_assign_CNN_WaferDetecting_2026_김준서

반도체 이미지 불량 검출 CNN 모델 개발1. 서론

최근 반도체 제조 공정은 공정 미세화와 생산성 향상으로 인해 불량 검출의 중요성이 더욱 커지고 있다. 웨이퍼 표면에 발생하는 다양한 결함은 반도체 소자의 수율과 신뢰성에 직접적인 영향을 미치므로, 생산 과정에서 결함을 빠르고 정확하게 검출하는 기술이 요구된다. 기존에는 작업자의 육안 검사나 규칙 기반의 영상처리 기법이 주로 사용되었으나, 이러한 방법은 다양한 결함 패턴을 정확하게 구분하는 데 한계가 있다.

최근에는 딥러닝 기반 영상 분류 기술이 발전하면서 CNN(Convolutional Neural Network)을 이용한 웨이퍼 결함 분류 연구가 활발히 이루어지고 있다. CNN은 이미지로부터 특징을 자동으로 추출하여 높은 분류 성능을 보이며, 다양한 결함 패턴을 효과적으로 구분할 수 있다는 장점을 가진다.

본 프로젝트에서는 **Original WM811K Silicon Wafer Map Image Dataset**을 이용하여 반도체 웨이퍼 이미지를 9개의 클래스로 분류하는 CNN 모델을 구현하였다. 또한 기본 CNN 모델을 설계한 후 Data Augmentation, Class Weight, Early Stopping 등의 기법을 적용하여 성능을 향상시키고자 하였다. 실험 과정에서 발생한 문제를 분석하고 해결하는 과정을 통해 최종적으로 높은 성능의 웨이퍼 불량 검출 모델을 구현하였다.

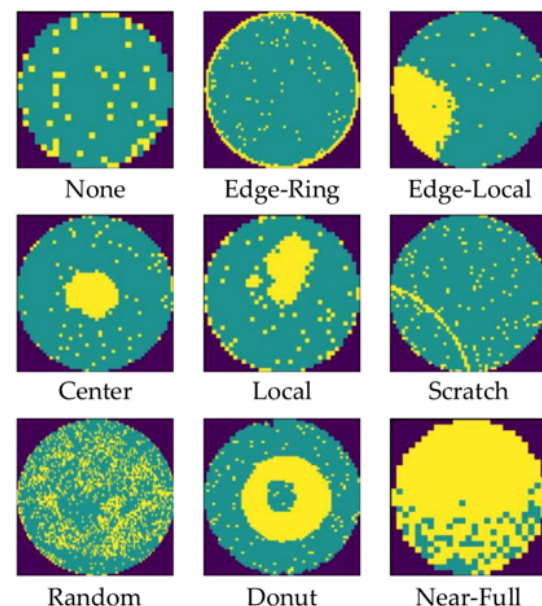
2. 데이터셋

본 프로젝트에서는 **Original WM811K Silicon Wafer Map Image Dataset**을 사용하였다. 해당 데이터셋은 실제 반도체 웨이퍼의 결함 이미지를 포함하고 있으며 다양한 결함 형태를 분류하기 위한 대표적인 데이터셋이다.

본 실험에서는 다음의 9개 클래스를 사용하였다.

Label	Class
0	Center
1	Donut
2	Edge-Loc
3	Edge-Ring
4	Loc
5	Near-full
6	Random
7	Scratch
8	none

(표 1) 클래스 목록



(그림 1) 데이터셋 예시

모든 이미지는 **32×32×3 RGB** 형태로 변환하여 입력 데이터로 사용하였다. 학습 데이터는 모델 학습에 사용하였으며, 테스트 데이터는 모델의 최종 성능을 평가하는 데 사용하였다.

데이터의 클래스 분포는 매우 불균형한 형태를 보였다. 특히 **none** 클래스는 약 **36,730**장의 학습 데이터와 약 **110,701**장의 테스트 데이터를 포함하여 다른 클래스보다 압도적으로 많은 비율을 차지하였다. 이러한 클래스 불균형은 모델이 특정 클래스에 편향되어 학습될 가능성을 높이므로, 학습 과정에서는 `compute_class_weight()` 를 이용하여 Class Weight를 적용하였다.

또한 학습 데이터는 `train_test_split()` 을 이용하여 **Train : Validation = 8 : 2** 비율로 분할하였으며, 각 클래스의 비율이 유지되도록 Stratified Split을 적용하였다.

Parameter	Value
Dataset	WM811K Origin
Input Size	32×32×3
Classes	9
Train / Validation	80 : 20
Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epoch	100
Loss	Categorical Crossentropy
Class Weight	Applied
EarlyStopping	Patience=15
ReduceLROnPlateau	Factor=0.5

(표 2) 실험 환경

3. CNN 모델 설계3.1 모델 구조

본 프로젝트에서는 웨이퍼 이미지의 다양한 결함 패턴을 효과적으로 추출하기 위해 Convolutional Neural Network(CNN)를 설계하였다. 모델은 총 3개의 Convolution Block과 1개의 Fully Connected Layer로 구성되며, 각 Block에서는 Convolution, Batch Normalization, Max Pooling 및 Dropout을 적절히 적용하였다.

초기 Layer에서는 저수준 특징(edge, texture 등)을 추출하고, 깊은 Layer에서는 결함의 형태와 위치 등 고수준 특징을 학습하도록 설계하였다. 마지막에는 Global Average Pooling을 적용하여 파라미터 수를 줄이고 과적합을 방지하였으며, Dense Layer와 Softmax 함수를 통해 최종적으로 9개의 클래스로 분류하였다.

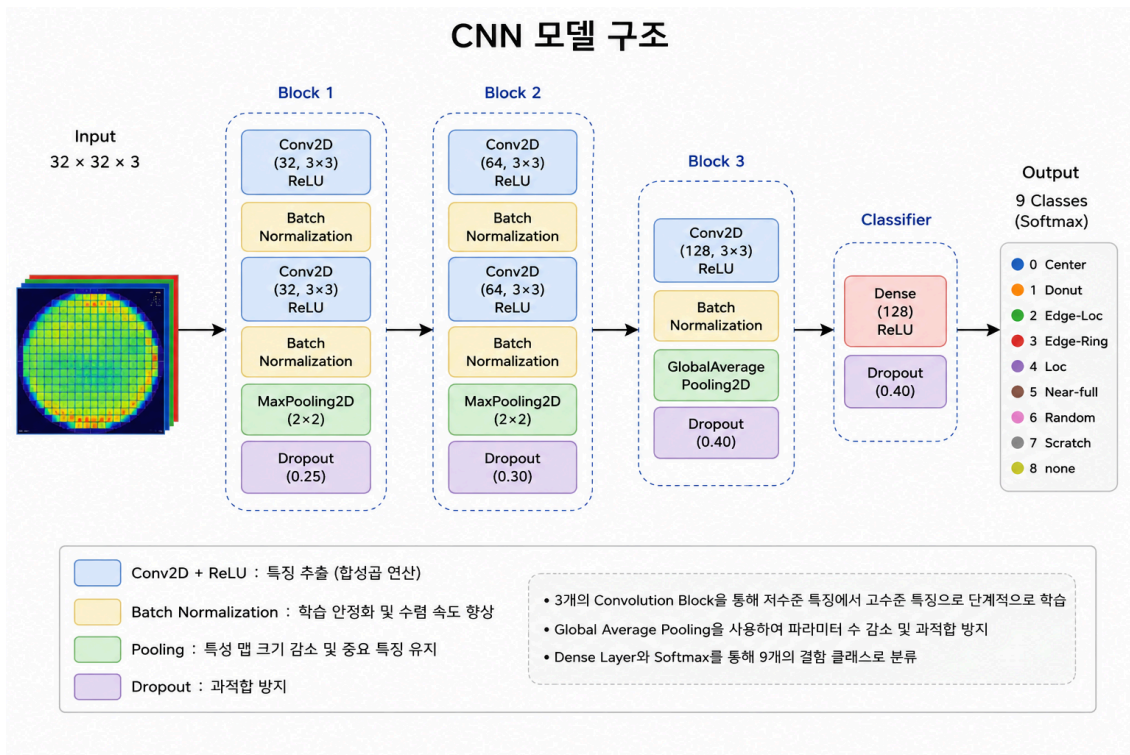


그림 2. CNN 모델 구조

3.2 모델 구성

본 모델의 전체 구조는 그림 2와 같으며 각 계층은 표 2와 같이 구성하였다.

Layer	Configuration
Input	32 × 32 × 3
Conv2D	32 Filters, 3×3, ReLU
Batch Normalization	-
Conv2D	32 Filters, 3×3, ReLU
Batch Normalization	-
MaxPooling	2 × 2
Dropout	0.25
Conv2D	64 Filters, 3×3, ReLU
Batch Normalization	-
Conv2D	64 Filters, 3×3, ReLU
Batch Normalization	-
MaxPooling	2 × 2
Dropout	0.30
Conv2D	128 Filters, 3×3, ReLU
Batch Normalization	-
Global Average Pooling	-
Dense	128, ReLU

Layer	Configuration
Dropout	0.40
Output	9 (Softmax)

표 3 CNN 모델 구조

3.3 설계 의도

CNN은 Convolution Layer를 통해 이미지의 공간적인 특징을 추출할 수 있기 때문에 웨이퍼 결함 검출에 적합하다.

첫 번째와 두 번째 Convolution Block에서는 작은 크기의 3×3 필터를 두 번 연속 사용하여 이미지의 세부적인 특징을 더욱 효과적으로 추출하도록 하였다. 이후 MaxPooling을 이용하여 Feature Map의 크기를 감소시키면서 계산량을 줄이고 중요한 특징만 유지하도록 하였다.

또한 각 Convolution Layer 뒤에는 Batch Normalization을 적용하였다. Batch Normalization은 학습 과정에서 Feature Distribution을 일정하게 유지하여 학습을 안정화시키고 수렴 속도를 향상시키는 역할을 한다.

Dropout은 과적합을 방지하기 위해 적용하였다. Feature가 증가할수록 Dropout 비율을 **0.25 → 0.30 → 0.40**으로 증가시켜 높은 수준의 Feature에 대한 과적합을 줄이고 일반화 성능을 향상시키도록 설계하였다.

마지막에는 Flatten 대신 **Global Average Pooling(GAP)**을 사용하였다. GAP는 Feature Map의 평균값만을 이용하여 Feature Vector를 생성하므로 파라미터 수를 크게 감소시키고 과적합을 줄이는 효과가 있다. 특히 웨이퍼 이미지와 같이 비교적 단순한 형태의 이미지에서는 Flatten보다 안정적인 성능을 기대할 수 있다.

출력층은 Softmax 함수를 사용하여 각 입력 이미지가 9개의 결함 클래스 중 어느 클래스에 속하는지를 확률 형태로 출력하도록 구성하였다.

3.4 학습 방법

학습에는 Adam Optimizer를 사용하였다. Adam은 Momentum과 Adaptive Learning Rate의 장점을 동시에 가지므로 CNN 학습에서 빠르고 안정적인 수렴 성능을 보인다.

또한 데이터의 클래스 불균형이 매우 심하기 때문에 `compute_class_weight()`를 이용하여 Class Weight를 적용하였다. 이를 통해 적은 수의 데이터를 가진 결함 클래스도 충분히 학습할 수 있도록 하였다.

학습이 과도하게 진행되어 과적합이 발생하는 것을 방지하기 위해 Early Stopping을 적용하였다. Validation Loss가 15 Epoch 동안 개선되지 않으면 학습을 종료하도록 설정하였으며, 가장 성능이 우수했던 모델의 가중치를 복원하도록 하였다.

Learning Rate는 초기값 **0.001**로 설정하였으며, Validation Loss가 개선되지 않는 경우 ReduceLROnPlateau를 사용하여 Learning Rate를 절반으로 감소시켰다. 이를 통해 초기에는 빠르게 학습하고 후반에는 보다 안정적으로 최적화에 수렴하도록 하였다.

4. 실험 결과 및 분석

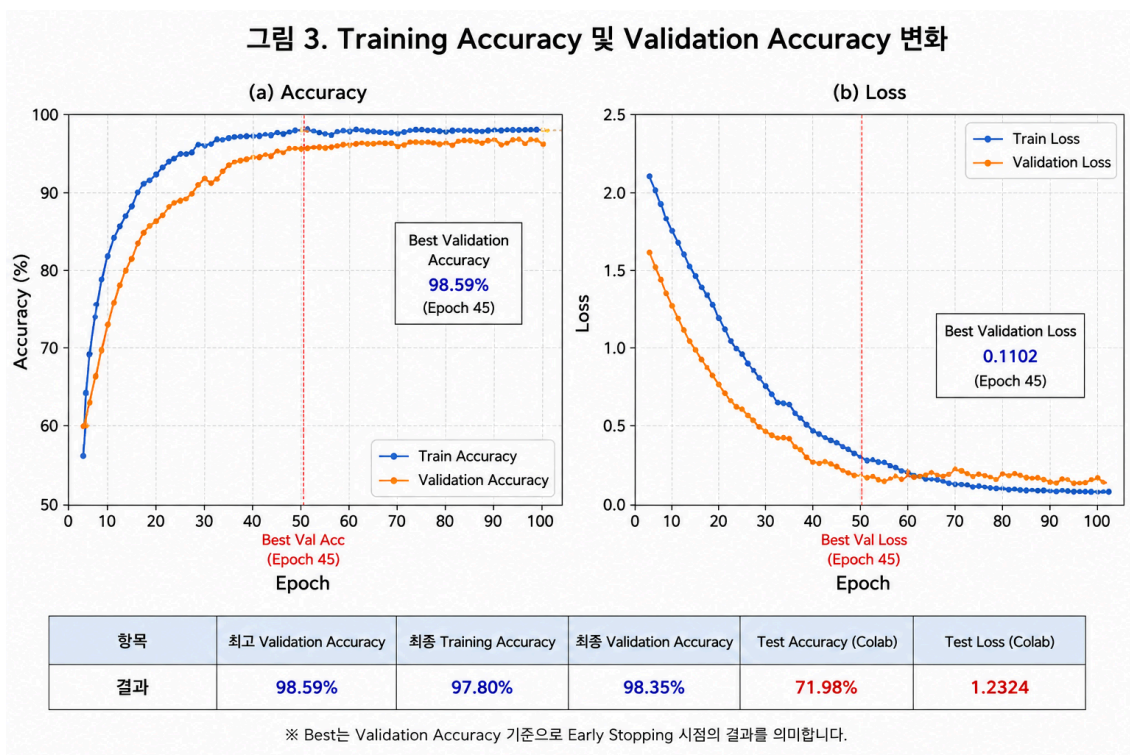
4.1 기본 CNN 모델 성능 평가

먼저 설계한 CNN 모델을 이용하여 기본 성능을 평가하였다. 학습은 Adam Optimizer를 사용하였으며, Learning Rate는 0.001, Batch Size는 32, Epoch는 최대 100으로 설정하였다. 또한 Class Weight를 적용하여 클래스 불균형 문제를 완화하였고, Early Stopping과 ReduceLROnPlateau를 사용하여 과적합을 방지하였다. 실험 조건은 표 4과 같다.

Parameter	Value
Input Size	32 × 32 × 3

Parameter	Value
Batch Size	32
Epoch	100
Optimizer	Adam
Learning Rate	0.001
Loss Function	Categorical Crossentropy
Validation Split	20% (Stratified)
Class Weight	Applied
EarlyStopping	Patience = 15
ReduceLROnPlateau	Factor = 0.5

표 4. 기본 CNN 모델 학습 조건



기본 CNN 모델의 학습 결과는 Validation Accuracy **98.59%**, Final Training Accuracy **97.80%**, Final Validation Accuracy **98.35%**를 기록하였다. 그러나 Test Accuracy는 **71.98%**로 Validation Accuracy에 비해 상대적으로 낮게 나타났다. Validation 성능은 매우 우수하였지만 Test 성능이 낮게 나타난 원인을 확인하기 위해 추가적인 실험을 진행하였다.

Metric	Result
Best Validation Accuracy	98.59%
Final Training Accuracy	97.80%
Final Validation Accuracy	98.35%
Test Accuracy	71.98%
Test Loss	1.2324

표 5. 기본 CNN 모델 성능

4.2 Data Augmentation 적용

Validation Accuracy와 Test Accuracy의 차이를 줄이고 일반화 성능을 향상시키기 위해 Data Augmentation을 적용하였다. Rotation, Width Shift, Height Shift, Horizontal Flip, Vertical Flip을 추가하였으며 나머지 학습 조건은 동일하게 유지하였다.

그러나 실험 결과 Test Accuracy는 ****66.77%****로 오히려 감소하였다. 단순한 Accuracy 비교만으로는 성능 저하의 원인을 확인하기 어려웠기 때문에 Confusion Matrix와 Classification Report를 추가로 분석하였다.

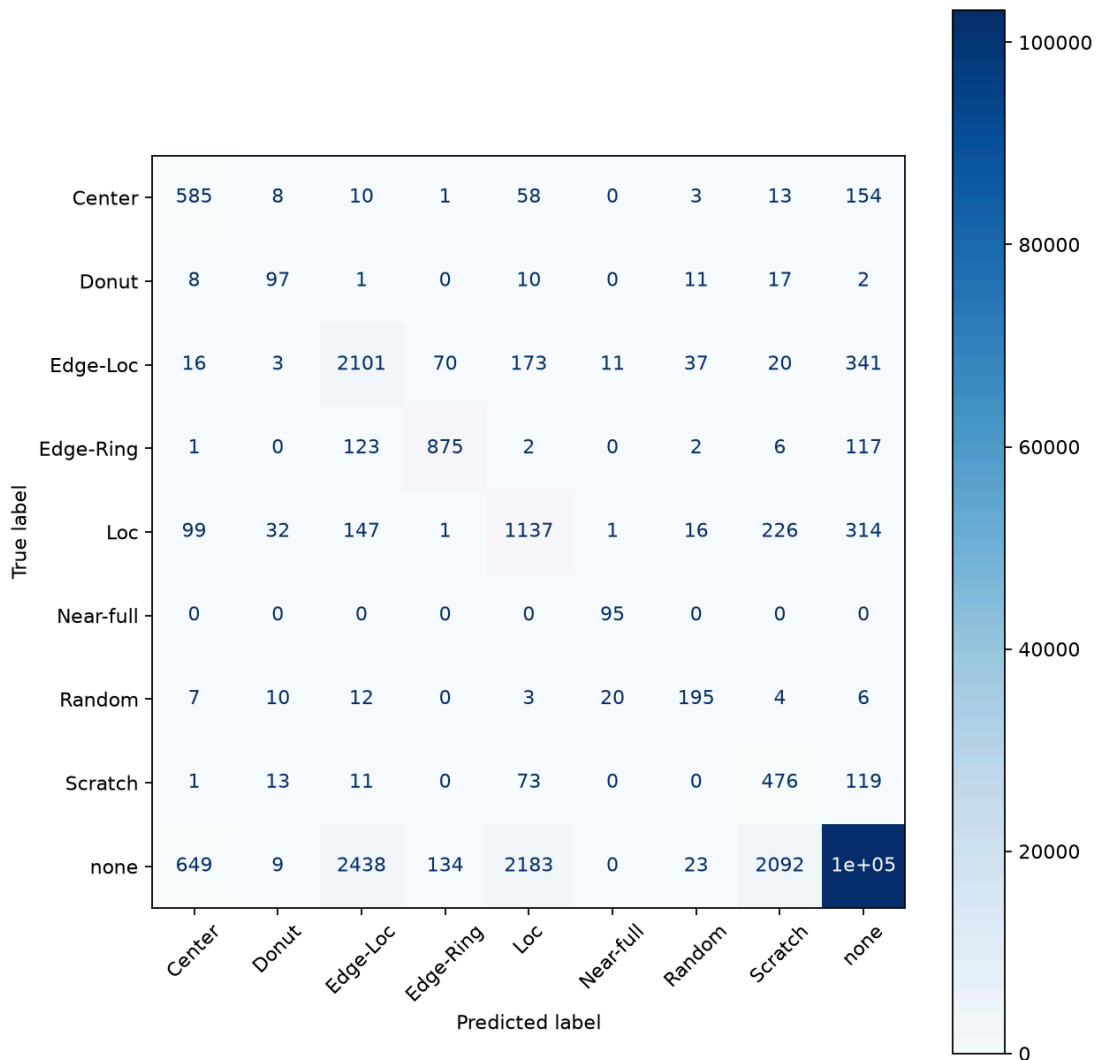
Metric	Result
Test Accuracy	66.77%
Test Loss	1.4125

표 6. 기본 CNN 모델 성능

4.3 Confusion Matrix 분석

Accuracy가 감소한 원인을 분석하기 위해 Confusion Matrix를 확인하였다. 분석 결과 예상과 다른 현상을 발견하였다. 모델이 **none 클래스를 잘못 분류한 것이 아니라, none 클래스 자체가 Confusion Matrix에서 나타나지 않았다.** 마지막 행의 support가 0으로 표시되었으며, 이는 none 클래스가 테스트 과정에서 정상적으로 평가되지 않았음을 의미하였다.

처음에는 Data Augmentation이 모델 성능을 저하시킨 것으로 판단하였으나, Confusion Matrix를 분석하면서 Accuracy 감소의 원인이 모델 구조가 아니라 **테스트 데이터 로딩 과정에 있을 가능성**을 확인하였다.



(그림 4. Data Augmentation 적용 후 Confusion Matrix)

4.4 원인 분석

테스트 데이터를 확인한 결과 none 클래스는 **약 110,701장의 이미지**를 포함하고 있었으며, 이는 다른 모든 클래스보다 압도적으로 많은 수였다. 반면 학습 데이터의 none 클래스는 약 36,730장이었다. 테스트 데이터의 none 클래스가 매우 큰 비중을 차지하면서 Google Colab 환경에서는 테스트 데이터를 모두 메모리에 로드하는 과정에서 런타임이 반복적으로 종료되었다.

이로 인해 테스트셋이 완전히 로드되지 못한 상태에서 성능 평가가 수행되었으며, 결과적으로 none 클래스가 제외된 상태의 Accuracy와 Confusion Matrix가 계산되었다. 즉, 초기에는 Data Augmentation이 성능을 저하시킨 것으로 판단하였지만, 실제 원인은 **실행 환경에서 발생한 테스트 데이터 로딩 문제**였다.

4.5 로컬 GPU 환경에서 재평가

Google Colab의 메모리 한계를 해결하기 위해 동일한 CNN 구조와 동일한 하이퍼파라미터를 유지한 상태에서 실행 환경만 **로컬 GPU(NVIDIA GeForce RTX 3050)** 환경으로 변경하였다. 로컬 환경에서는 테스트 데이터를 정량적으로 모두 로드할 수 있었으며, none 클래스를 포함한 전체 테스트셋을 대상으로 다시 성능을 평가하였다.

재평가 결과 Test Accuracy는 **91.69%**, Test Loss는 **0.2728**로 크게 향상되었다. Validation Accuracy 역시 ****98.61%****를 유지하였으며, Google Colab에서 측정한 71.98%는 모델의 성능 한계가 아니라 테스트 데이터가 완전히 로드되지 못한 환경적 문제였음을 확인하였다.

Metric	Result
Best Validation Accuracy	98.61%
Final Training Accuracy	98.02%
Final Validation Accuracy	98.61%
Test Accuracy	91.69%
Test Loss	0.2728

표 7. 로컬 GPU 환경 최종 성능

4.6 결과 분석

최종 Confusion Matrix와 Classification Report를 분석한 결과 **none** 클래스는 **Precision 0.99, Recall 0.93, F1-score 0.96**으로 매우 높은 성능을 보였다. 또한 Edge-Ring과 Near-full 클래스 역시 높은 분류 정확도를 나타냈다. 반면 Center, Edge-Loc, Loc, Scratch 클래스는 서로 유사한 패턴을 가지므로 일부 오분류가 발생하였다. 이는 결함 형태가 비슷한 클래스 간의 특징이 CNN에서 완전히 분리되지 않았기 때문으로 판단된다.

이번 실험을 통해 모델 자체뿐 아니라 **실행 환경과 데이터 로딩 과정도 성능 평가에 큰 영향을 미칠 수 있음**을 확인하였다. 따라서 대용량 이미지 데이터셋을 사용하는 경우에는 모델 구조뿐 아니라 데이터 처리 과정과 실행 환경 또한 함께 고려해야 함을 알 수 있었다.

5. 결론

본 프로젝트에서는 Original WM811K Silicon Wafer Map Image Dataset을 이용하여 반도체 웨이퍼 불량률 분류하는 CNN 모델을 구현하였다. Conv2D, Batch Normalization, MaxPooling, Dropout을 이용한 CNN 구조를 설계하고 Adam Optimizer, Class Weight, Early Stopping 및 ReduceLROnPlateau를 적용하여 모델을 학습하였다.

기본 CNN 모델은 Validation Accuracy ****98.59%****를 달성하였으나 Test Accuracy는 ****71.98%****로 나타나 Validation 성능과 큰 차이를 보였다. 이후 일반화 성능 향상을 위해 Data Augmentation을 적용하였으나 Test Accuracy는 오히려 ****66.77%****로 감소하였다. 이에 Accuracy만으로 성능을 판단하지 않고 Confusion Matrix와 Classification Report를 함께 분석하였다.

분석 과정에서 none 클래스가 테스트 결과에 포함되지 않는 비정상적인 현상을 확인하였으며, 원인을 조사한 결과 Google Colab 환경에서 대용량 테스트 데이터(none 클래스 약 110,701장)를 모두 로드하지 못하여 런타임이 종료되는 문제가 발생했음을 확인하였다. 이는 모델 구조의 문제가 아니라 실행 환경의 한계로 인해 발생한 문제였다.

동일한 CNN 모델과 동일한 학습 조건을 유지한 상태에서 로컬 GPU 환경(NVIDIA GeForce RTX 3050)에서 다시 실험을 수행한 결과, 전체 테스트 데이터를 정상적으로 평가할 수 있었으며 최종적으로 **91.69%의 Test Accuracy**를 달성하였다. 이를 통해 초기 성능 저하가 모델 자체의 한계가 아니라 테스트 데이터 로딩 과정에서 발생한 문제였음을 확인하였다.

이번 프로젝트를 수행하면서 CNN 모델의 설계와 학습뿐만 아니라 데이터 전처리, 클래스 불균형 처리, 성능 평가 방법, 실행 환경에 따른 차이까지 종합적으로 경험할 수 있었다. 특히 단순히 Accuracy만 확인하는 것이 아니라 Confusion Matrix와 Classification Report를 함께 분석하여 문제의 원인을 파악하는 과정의 중요성을 확인하

였다. 또한 대용량 데이터셋에서는 모델의 성능뿐 아니라 데이터 처리 과정과 실행 환경 역시 최종 결과에 큰 영향을 미친다는 점을 경험하였다.

향후에는 ResNet, EfficientNet과 같은 Pre-trained Network를 적용하거나 Attention Mechanism을 추가하여 유사한 결함 패턴 간의 분류 성능을 향상시키는 연구를 수행한다면 더욱 높은 정확도를 기대할 수 있을 것으로 판단된다.